

Jaypee Institute of Information Technology, Noida

Department of Computer Applications



PROJECT-BASED LEARNING REPORT

Near-Earth Object (NEO) Analysis System

ETL Pipeline and SQL-Based Scientific Data Analysis

Submitted by:

Bhavninder Singh Virk 992509260003

Submitted for:

Course Name: Data Management Using SQL

Program: MSc in Data and Business Analytics

Submitted to: Prof. Dr. Devpriya Soni

Academic Year: 2025–26

TABLE OF CONTENTS

ABSTRACT	3
1. INTRODUCTION.....	3
2. PROBLEM STATEMENT	4
2.1 Core Challenges Addressed	4
3. DATASET OVERVIEW	4
4. DATA ACQUISITION (NASA API)	5
4.1 Extraction Process	5
4.2 Sample Python Extraction Script	5
5. DATABASE DESIGN & NORMALIZATION	6
5.1 Normalization Rationale	6
5.2 Entity-Relationship Overview	6
5.3 Table Descriptions	6
5.4 DDL — Table Creation (MySQL)	6
6. SQL ANALYSIS & QUERY IMPLEMENTATION.....	7
6.1 Basic Descriptive Queries	7
6.1.1 Total Asteroid Count	7
6.1.2 Hazardous vs Non-Hazardous Distribution.....	7
6.2 Join Operations — Full Combined Analysis	7
6.3 Advanced Analytical Queries	7
6.3.1 Closest Approach Ranking.....	7
6.3.2 Fastest Asteroids	7
6.3.3 Size Categorization Using CASE.....	8
6.3.4 Kinetic Risk Index — Custom Composite Metric	8
6.4 View Creation — High-Speed Threat Dashboard	8
7. ANALYTICAL RESULTS.....	9
7.1 Hazard Classification Summary	9
7.2 Velocity Analysis	9
7.3 Key Analytical Findings	9
8. PERFORMANCE AND OPTIMIZATION	9
8.1 Index Creation	9
8.2 Performance Impact Summary	10
9. REAL-WORLD RELEVANCE AND APPLICATIONS	10
9.1 Scientific and Defense Applications.....	10
9.2 Commercial and Data Engineering Applications	10
10. CONCLUSION	11
REFERENCES.....	12

ABSTRACT

This project presents a complete data pipeline for analyzing Near-Earth Objects (NEOs) using real-time data from NASA's NeoWs API. The system extracts asteroid data over a 30-day period, transforms raw JSON into a structured format, and loads it into a normalized MySQL database.

The database is designed using relational modeling principles by separating static asteroid attributes from dynamic observation data. Advanced SQL queries are implemented to generate insights such as velocity trends, proximity analysis, hazard classification, and risk modeling.

A custom metric, the Kinetic Risk Index, is introduced to estimate potential impact severity. The system demonstrates practical applications of ETL pipelines, database normalization, and analytical querying in scientific data processing.

Keywords: *ETL Pipeline, NASA API, Near-Earth Objects, SQL, Data Analysis, Normalization, Risk Modeling, Kinetic Risk Index, MySQL.*

1. INTRODUCTION

Near-Earth Objects (NEOs) are asteroids and comets that approach within 1.3 AU of the Sun, occasionally crossing Earth's orbital path. Monitoring these objects is critical for planetary defense, scientific research, and understanding the composition of the early solar system.

This project focuses on building a structured, end-to-end data system to:

- Collect real-time asteroid data via NASA's public NeoWs API
- Transform raw JSON responses into clean, structured tabular records
- Store data efficiently using a normalized relational database in MySQL
- Analyze the dataset using advanced SQL queries to generate actionable insights

The project integrates concepts from API-based data extraction, data transformation and cleaning, database normalization, and analytical SQL querying. Together, these form a complete, production-style data pipeline applicable to both scientific and business domains.

2. PROBLEM STATEMENT

Raw astronomical data retrieved from APIs presents significant analytical challenges. JSON-formatted API responses are unstructured and cannot be queried directly with SQL, the data is not optimized for storage efficiency or relational analysis, and performing multi-dimensional analysis (velocity, size, risk) requires a normalized schema with well-defined relationships.

2.1 Core Challenges Addressed

- Converting nested JSON API responses into flat, structured tabular format
- Designing a normalized relational database to minimize redundancy
- Extracting meaningful scientific and risk-related insights using SQL
- Defining a custom composite metric (Kinetic Risk Index) for hazard ranking

3. DATASET OVERVIEW

The dataset is collected dynamically from NASA's NeoWs (Near-Earth Object Web Service) API. Each record represents a single close-approach observation of an asteroid, including physical characteristics and approach parameters.

Column	Description
asteroid_id	Unique NASA identifier for each asteroid
name	Official asteroid designation or name
absolute_magnitude	Intrinsic brightness measure (H magnitude)
is_hazardous	Boolean flag: potentially hazardous (1) or not (0)
diameter_min_km	Minimum estimated diameter in kilometres
diameter_max_km	Maximum estimated diameter in kilometres
velocity_kph	Relative velocity at closest approach (km/h)
miss_distance_km	Distance of closest approach from Earth (km)

Table 1: Dataset Attributes from NASA NeoWs API

4. DATA ACQUISITION (NASA API)

Data was extracted using NASA's NeoWs REST API, which provides asteroid observation data for any specified date range. A Python script was developed to automate collection across a 30-day window, parse the JSON response, and export the structured data to CSV for database loading.

4.1 Extraction Process

- Python script developed using the requests library for HTTP access
- NASA developer API key used for authenticated access
- Data collected across a 30-day date range
- Nested JSON parsed and flattened into tabular records
- Cleaned dataset exported to asteroid_data.csv for SQL import

4.2 Sample Python Extraction Script

```
import requests
import pandas as pd

API_KEY = "YOUR_API_KEY"
url = f"https://api.nasa.gov/neo/rest/v1/feed?api_key={API_KEY}"

response = requests.get(url)
data = response.json()

asteroids = []
for date in data['near_earth_objects']:
    for obj in data['near_earth_objects'][date]:
        asteroids.append({
            "id": obj["id"],
            "name": obj["name"],
            "magnitude": obj["absolute_magnitude_h"],
            "is_hazardous": obj["is_potentially_hazardous_asteroid"],
            "diameter_min_km":
obj["estimated_diameter"]["kilometers"]["estimated_diameter_min"],
            "diameter_max_km":
obj["estimated_diameter"]["kilometers"]["estimated_diameter_max"],
            "velocity_kph":
obj["close_approach_data"][0]["relative_velocity"]["kilometers_per_hour"],
            "miss_distance_km":
obj["close_approach_data"][0]["miss_distance"]["kilometers"]
        })

df = pd.DataFrame(asteroids)
df.to_csv("asteroid_data.csv", index=False)
```

5. DATABASE DESIGN & NORMALIZATION

To ensure efficient storage and eliminate redundancy, the dataset is normalized into two relational tables following First, Second, and Third Normal Form (1NF, 2NF, 3NF) principles. Static asteroid attributes that do not change between approaches are separated from dynamic approach-specific measurements.

5.1 Normalization Rationale

A single denormalized table would repeat static attributes (name, magnitude, hazard flag) for every observation of the same asteroid. Separating these into a parent table and a child table:

- Eliminates data redundancy for repeated asteroid sightings
- Ensures update anomalies are prevented (one row to update per asteroid)
- Establishes a clear one-to-many relationship between registry and approaches

5.2 Entity-Relationship Overview

```
asteroid_registry (1) —< asteroid_approaches (M)
```

One asteroid may have multiple close-approach observation records.

5.3 Table Descriptions

Table	Key	Purpose / Columns
asteroid_registry	PK: asteroid_id	Static attributes: name, absolute_magnitude, is_hazardous
asteroid_approaches	PK: approach_id FK: asteroid_id	Dynamic data: diameter_min_km, diameter_max_km, velocity_kph, miss_distance_km

Table 2: Normalized Schema — 2 Relational Tables

5.4 DDL — Table Creation (MySQL)

```
-- PARENT TABLE: static asteroid attributes
CREATE TABLE asteroid_registry (
  asteroid_id      VARCHAR(20)    PRIMARY KEY,
  name             VARCHAR(200)   NOT NULL,
  absolute_magnitude DECIMAL(6,2),
  is_hazardous     TINYINT(1)     NOT NULL DEFAULT 0
);

-- CHILD TABLE: dynamic per-approach observations
CREATE TABLE asteroid_approaches (
  approach_id      INT             AUTO_INCREMENT PRIMARY KEY,
  asteroid_id       VARCHAR(20)    NOT NULL,
  diameter_min_km   DECIMAL(10,6),
```

```
diameter_max_km    DECIMAL(10,6),
velocity_kph       DECIMAL(15,4),
miss_distance_km   DECIMAL(15,4),
FOREIGN KEY (asteroid_id) REFERENCES asteroid_registry(asteroid_id)
);
```

6. SQL ANALYSIS & QUERY IMPLEMENTATION

A comprehensive suite of SQL queries is implemented, ranging from basic descriptive statistics to advanced risk modeling. All queries join the two normalized tables to produce complete analytical results.

6.1 Basic Descriptive Queries

6.1.1 Total Asteroid Count

```
SELECT COUNT(*) AS total_asteroids FROM asteroid_registry;
```

6.1.2 Hazardous vs Non-Hazardous Distribution

```
SELECT
    is_hazardous,
    COUNT(*) AS count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM asteroid_registry), 2) AS pct
FROM asteroid_registry
GROUP BY is_hazardous;
```

6.2 Join Operations — Full Combined Analysis

Both tables are combined using an INNER JOIN to enable multi-dimensional analysis across static and dynamic attributes:

```
SELECT r.name, r.is_hazardous, a.velocity_kph, a.miss_distance_km,
       a.diameter_min_km, a.diameter_max_km
FROM asteroid_registry r
JOIN asteroid_approaches a ON r.asteroid_id = a.asteroid_id
ORDER BY a.miss_distance_km ASC;
```

6.3 Advanced Analytical Queries

6.3.1 Closest Approach Ranking

```
SELECT r.name, a.miss_distance_km
FROM asteroid_registry r
JOIN asteroid_approaches a ON r.asteroid_id = a.asteroid_id
ORDER BY a.miss_distance_km ASC
LIMIT 10;
```

6.3.2 Fastest Asteroids

```
SELECT r.name, a.velocity_kph
FROM asteroid_registry r
JOIN asteroid_approaches a ON r.asteroid_id = a.asteroid_id
ORDER BY a.velocity_kph DESC
LIMIT 10;
```

6.3.3 Size Categorization Using CASE

```
SELECT r.name,
       CASE
         WHEN a.diameter_max_km < 0.05 THEN 'Small (< 50 m)'
         WHEN a.diameter_max_km < 0.20 THEN 'Medium (50-200 m)'
         ELSE                             'Large (> 200 m)'
       END AS size_category,
       a.diameter_max_km
FROM asteroid_registry r
JOIN asteroid_approaches a ON r.asteroid_id = a.asteroid_id
ORDER BY a.diameter_max_km DESC;
```

6.3.4 Kinetic Risk Index — Custom Composite Metric

A custom metric is introduced to estimate the relative potential impact severity of each asteroid. The index is computed as the product of the average estimated diameter and the approach velocity, providing a risk score that increases with both size and speed:

```
-- Kinetic Risk Index = avg_diameter_km * velocity_kph
SELECT
  r.name,
  r.is_hazardous,
  ROUND((a.diameter_min_km + a.diameter_max_km) / 2, 6) AS avg_diameter_km,
  a.velocity_kph,
  ROUND(((a.diameter_min_km + a.diameter_max_km) / 2) * a.velocity_kph, 2)
    AS kinetic_risk_index
FROM asteroid_registry r
JOIN asteroid_approaches a ON r.asteroid_id = a.asteroid_id
ORDER BY kinetic_risk_index DESC
LIMIT 20;
```

6.4 View Creation — High-Speed Threat Dashboard

A reusable view is defined to isolate the highest-risk observations for ongoing monitoring. This view filters for potentially hazardous asteroids travelling above 50,000 km/h that passed within 1,000,000 km of Earth:

```
CREATE VIEW high_speed_threats AS
SELECT
  r.name,
  r.is_hazardous,
  a.velocity_kph,
  a.miss_distance_km,
  ROUND(((a.diameter_min_km + a.diameter_max_km) / 2) * a.velocity_kph, 2)
    AS kinetic_risk_index
FROM asteroid_registry r
JOIN asteroid_approaches a ON r.asteroid_id = a.asteroid_id
WHERE r.is_hazardous = 1
      AND a.velocity_kph > 50000
      AND a.miss_distance_km < 1000000
ORDER BY kinetic_risk_index DESC;
```


7. ANALYTICAL RESULTS

Key insights were derived from the 30-day NEO dataset. The findings below reflect representative patterns observed across the full observation period.

7.1 Hazard Classification Summary

Category	Count (Approx.)	Observation
Non-Hazardous	~85%	Majority of observed NEOs
Potentially Hazardous	~15%	Flagged for priority monitoring

Table 3: Hazard Classification Distribution

7.2 Velocity Analysis

Metric	Non-Hazardous	Hazardous
Average Velocity (km/h)	~42,000	~61,000
Maximum Velocity (km/h)	~95,000	~115,000

Table 4: Velocity Comparison — Hazardous vs Non-Hazardous Asteroids

7.3 Key Analytical Findings

- Potentially hazardous asteroids are fewer in number but consistently travel at higher velocities than non-hazardous ones.
- Larger asteroids do not always exhibit higher velocity; size and speed are not strongly correlated in this dataset.
- The closest-approach objects are not always the highest-risk; small, fast objects with high Kinetic Risk Index scores warrant equal attention.
- Objects in the Small category (< 50 m diameter) are disproportionately numerous but carry low individual risk scores.

8. PERFORMANCE AND OPTIMIZATION

SQL query optimization is essential for analytical workloads on growing asteroid datasets. The following indexing strategy was applied to minimize JOIN overhead and accelerate common filter operations.

8.1 Index Creation

```
-- Primary join index on the foreign key
CREATE INDEX idx_asteroid_id ON asteroid_approaches(asteroid_id);

-- Filter acceleration for hazard-based queries
```

```
CREATE INDEX idx_is_hazardous ON asteroid_registry(is_hazardous);

-- Range query support for velocity and distance filters
CREATE INDEX idx_velocity      ON asteroid_approaches(velocity_kph);
CREATE INDEX idx_miss_distance ON asteroid_approaches(miss_distance_km);
```

8.2 Performance Impact Summary

Optimization	Technique	Impact	Best For
Foreign Key Index	idx_asteroid_id	Faster JOIN resolution	All multi-table queries
Hazard Filter Index	idx_is_hazardous	Skip non-hazardous rows	Risk dashboard queries
Velocity Range Index	idx_velocity	Range scan vs full scan	Top-N speed queries
Distance Range Index	idx_miss_distance	Fast proximity ranking	Closest-approach queries

Table 5: Indexing Strategy and Performance Impact

9. REAL-WORLD RELEVANCE AND APPLICATIONS

The techniques demonstrated in this project extend well beyond asteroid monitoring. The same ETL pipeline and SQL analysis framework is directly applicable to a wide range of scientific and business domains.

9.1 Scientific and Defense Applications

- **NASA's Planetary Defense Coordination Office and ESA's Hera mission rely on structured asteroid databases for risk assessment and mission planning.** Planetary Defense:
- **Normalized relational databases allow rapid querying to support mission briefings and congressional reports on NEO hazards.** Space Agency Reporting:
- **Astronomers can use structured SQL-accessible datasets to study orbital mechanics, composition trends, and historical close approaches.** Research Institutions:

9.2 Commercial and Data Engineering Applications

- **The extract-transform-load workflow demonstrated here is the foundation of enterprise data engineering in tools such as Apache Airflow, dbt, and AWS Glue.** ETL Pipeline Development:
- **The Kinetic Risk Index composite metric mirrors techniques used in financial risk scoring, insurance underwriting, and supply-chain risk models.** Risk Assessment Systems:

- **Many business intelligence systems consume REST API data in JSON format; this project demonstrates the full pipeline from raw JSON to queryable relational tables.** API Data Integration:

10. CONCLUSION

This project successfully demonstrates a complete, production-style data pipeline applied to the scientific domain of planetary defense. The key outcomes are summarized below:

- API-based data extraction was automated using Python and the NASA NeoWs REST API, collecting 30 days of real-time asteroid observation data.
- Data transformation produced a clean, structured dataset from nested JSON responses, ready for relational database loading.
- Database normalization separated static asteroid attributes from dynamic approach records, implementing 1NF, 2NF, and 3NF principles across two related tables.
- Advanced SQL analysis delivered insights into velocity trends, proximity ranking, size distribution, and hazard classification using JOIN operations, CASE expressions, aggregate functions, and views.
- The Kinetic Risk Index, a custom composite metric, provides a single comparable score for ranking potential impact severity across all observed asteroids.
- Performance optimization through targeted indexing on foreign keys and filter columns ensures the system scales effectively as the dataset grows.

Best Practice Summary: *Always normalize before analyzing. Use views to encapsulate complex risk queries for reuse. Index all foreign key and high-cardinality filter columns. Validate API data before loading into a production database.*

REFERENCES

- [1] NASA Open APIs. "Near-Earth Object Web Service (NeoWs)." [Online]. Available: <https://api.nasa.gov/>
- [2] NASA Jet Propulsion Laboratory. "Center for Near-Earth Object Studies." [Online]. Available: <https://cneos.jpl.nasa.gov/>
- [3] Python Software Foundation. "Requests: HTTP for Humans." [Online]. Available: <https://docs.python-requests.org/>
- [4] Oracle Corporation. MySQL 8.0 Reference Manual, 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [5] Pandas Documentation. "Pandas: Data Analysis Library." [Online]. Available: <https://pandas.pydata.org/>
- [6] Ramakrishnan, R. and Gehrke, J., Database Management Systems, 3rd ed. McGraw-Hill, 2002.
- [7] Kimball, R. and Ross, M., The Data Warehouse Toolkit, 3rd ed. Wiley, 2013.
- [8] Date, C. J., An Introduction to Database Systems, 8th ed. Addison-Wesley, 2004.